

732A94 Advanced R programming

Lecture 3

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson and Krzysztof Bartoszek)

Linköpings Universitet

- Scientific computing: best practices
- R packages
- Git and GitHub
- Creating R packages
- Documentation with roxygen2
- Unit testing with testthat
- Debugging

Questions?

Any questions from last time?

Based on the article referred to on the course page (Wilson et al., *Best Practices for Scientific Computing*).

Based on the article referred to on the course page (Wilson et al., *Best Practices for Scientific Computing*).

REMEMBER

1. Nothing replaces thinking and predicting.
2. Crystal balls do not exist.
3. Best practices do not work if used without thinking.

1. Write code for people

- A program should not require its readers to hold more than a handful of facts in memory at once
- Make names consistent, distinctive and meaningful
- Make code style and formatting consistent

Naming conventions:

- `camelCase`
- `PascalCase`
- `snake_case` (the R and Python convention)
- `kebab-case`
- avoid `dot.case` – in R the dot means something to S3 dispatch!

`dot.case` is genuinely dangerous in R: `print.myclass` is *not* just a name, it is an S3 method for class `myclass`.

```
# Tools that keep style consistent for you  
styler::style_file("R/my_function.R") # reformats  
lintr::lint_package() # complains
```

Follow one style guide, e.g. <https://style.tidyverse.org/>.

2. Let the computer do the work

- Make the computer repeat tasks
- Save recent commands in a file for re-use (**.Rhistory**)
- Use a build tool to automate workflows

If you have run it twice by hand, write a script.

3. Make incremental changes

- Work in small steps with frequent feedback and course correction
- Use a version control system
- Put *everything that has been created manually* in version control

4. Do not repeat yourself (or others)

- Every piece of data must have a single authoritative representation in the system
- Modularize code rather than copying and pasting
- Re-use code instead of rewriting it

5. Plan for mistakes

- Add assertions to programs to check their operation
- Use an off-the-shelf unit testing library
- Turn bugs into test cases
- Use a symbolic debugger

ALWAYS CHECK CORRECTNESS OF INPUT! (also on the exam...)

Checking input

```
my_mean <- function(x, na.rm = FALSE) {  
  stopifnot(is.numeric(x), is.logical(na.rm),  
            length(na.rm) == 1)  
  sum(x, na.rm = na.rm) / length(x)  
}  
my_mean(c(1, 2, 3))
```

```
## [1] 2
```

```
my_mean("a")
```

```
## Error in `my_mean()`:  
## ! is.numeric(x) is not TRUE
```

Checking input

`match.arg()` validates *and* documents the allowed values:

```
scale_data <- function(x, method = c("z", "minmax")) {  
  method <- match.arg(method)  
  switch(method,  
    z = (x - mean(x)) / sd(x),  
    minmax = (x - min(x)) / diff(range(x)))  
}  
scale_data(1:5)
```

```
## [1] -1.2649111 -0.6324555  0.0000000  0.6324555  1.2649111
```

```
scale_data(1:5, "nonsense")
```

```
## Error in `match.arg()`:
```

```
## ! 'arg' should be one of "z", "minmax"
```

6. Optimize software only after it works correctly

- Use a profiler to identify bottlenecks
- Write code in the highest-level language possible

But prepare the code for the optimal algorithm... (more in Lecture 6)

7. Document design and purpose, not mechanics

- Document interfaces and reasons, not implementations
(but make sure that *you* are able to understand the implementation)
- Refactor code in preference to explaining how it works
- Embed the documentation for a piece of software in that software

8. Collaborate

- Use pre-merge code reviews
- Use pair programming when bringing someone new up to speed and when tackling particularly tricky problems
- Use an issue tracking tool

An environment with functions and/or data.

The way to share code and data.

Number of packages on CRAN:

- 4 May 2018 – roughly 12 500 packages
- 15 August 2022 – 18 428 packages
- 1 September 2026 – 24 801 packages

```
nrow(available.packages(  
  available_packages_filters = c("CRAN")))
```

Usage

- `library()`
- `::`
- `:::`

Installation

- `install.packages()`
- `devtools::install_github()` (Recently deprecated works for now)
- `devtools::install_local()` (Recently deprecated works for now)
- `pak::pkg_install()` The new way!

Package namespace

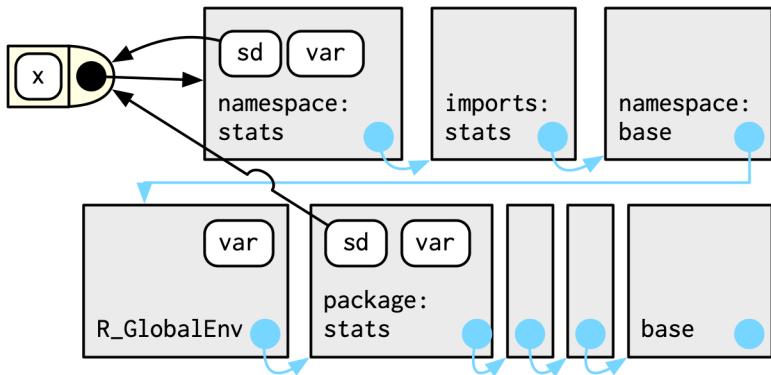


Figure 1: Package namespace (H. Wickham, Adv. R)

- `library()`: **attaches** a package, its functions become available on the search path
- `requireNamespace()`: **loads** the package's code, data and methods and runs `.onLoad()`. The package is available in memory but is *not* on the search path – access its components with `::`

NEVER use `library()` or `require()` inside your own package, CRAN forbids it.

H. Wickham, *R packages* (p. 82–84)

Which are good packages?

Examine the package!

1. Who is the author?
2. When was it last updated?
3. Is it in development?

```
packageVersion("ggplot2")  
packageDescription("ggplot2")
```

Version numbering [major].[minor].[patch]

Why version control?

When working alone we can each manage our work the way we prefer. It gets hairy when we must collaborate:

1. You and a colleague write different modules. You changed something this morning that your colleague expected to remain unchanged.
2. A stakeholder dislikes a recent change and wants a rollback. Have you stored previous versions? Can you retrieve them easily?
3. Your main server just crashed. Do you have a backup?
4. Version control is ubiquitous – your recruiter probably wants you to have version control skills.

1. Simple to use
2. Distributed
 - No single copy of the work product
 - Each collaborator has all the files and the change history
3. Fast
4. Common in practice
5. R packages use GitHub
6. Integrated with RStudio

1. Simple to use
2. Distributed
 - No single copy of the work product
 - Each collaborator has all the files and the change history
3. Fast
4. Common in practice
5. R packages use GitHub
6. Integrated with RStudio

Created by Linus Torvalds! ;)

Git: the lifecycle of files

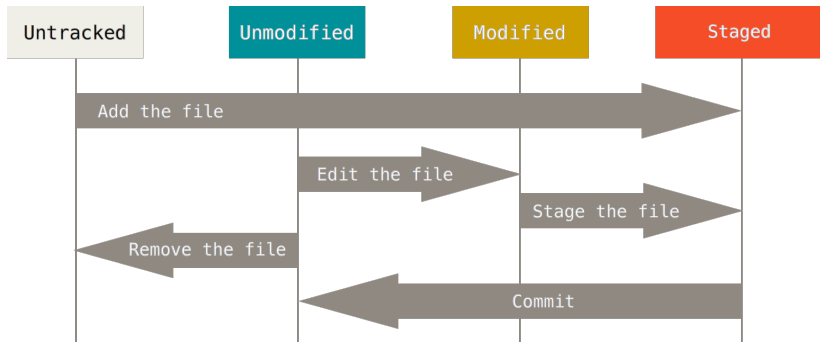


Figure 2: The lifecycle of file status (Chacon & Straub, *Pro Git*)

Chapters 1 and 2 of *Pro Git* are highly recommended if Git is new to you.

1. A *repository* (repo) is where the files of your project reside
2. A newly created file is by default **untracked**
3. To track a file, **add** it to the repo
4. **Commit** an added file to save it in the repo; the file state becomes **unmodified**
5. Editing an unmodified file changes its state to **modified**
6. To save your edits, stage and commit the file again

Git: the commands you need

```
git clone https://github.com/user/repo.git
git status          # what is going on?
git add R/my_fun.R  # stage
git commit -m "Add my_fun()"
git pull            # fetch + merge from remote
git push            # send commits to remote
git log --oneline --graph
```

Commit messages are documentation – write them for people.

1. Local (commit)
2. Remote (push/pull)
3. Barebones homepage (using markdown)
4. Collaboration
5. Issue tracker / wiki / discussions

Public and private repos.

2FA: time-based one-time passwords, open algorithm, value based on an initial key and the current time (R package **otp**).

A typical collaborative workflow

```
git switch -c feature/euclidean  # new branch  
# ... edit, add, commit ...  
git push -u origin feature/euclidean
```

Then open a **pull request** on GitHub, let a group member review it, and merge.

Branches are cheap. Use one per feature – **main** should always work.

1. Competing commits from different contributors
2. Git cannot resolve this by itself
3. Resolve manually
4. Supporting merge tools

Conflicts are normal, not a sign that something went wrong.

What a conflict looks like

Git edits the file and leaves **conflict markers** in it:

```
<<<<<< HEAD
euclidean <- function(a, b) { ... your version ... }
=====
euclidean <- function(x, y) { ... their version ... }
>>>>>> feature/euclidean
```

- Everything between <<<<<< and ===== is *your* version
- Everything between ===== and >>>>>> is *theirs*

Resolving a conflict

1. Open the conflicting file in an editor
2. Delete the conflict markers and choose which version (or a combination) to keep
3. Stage and commit

```
git status          # lists "both modified" files
# ... edit the file ...
git add euclidean.R
git commit          # completes the merge
git merge --abort    # escape hatch: start over
```

You do not have to edit the markers by hand:

- RStudio and VS Code both ship a built-in three-way merge view
- `git mergetool` with e.g. `meld`, `kdiff3`, `p4merge`
- Sublime Merge, <https://www.sublimemerge.com/>

```
git config --global merge.tool meld
git mergetool
```

.gitignore for R projects

Do not commit generated files or secrets:

```
.Rproj.user  
.Rhistory  
.RData  
.Renviron  
*.Rcheck/  
*.tar.gz  
src/*.o  
src/*.so
```

`usethis::git_vaccinate()` sets up a sensible global ignore for you.

Why are packages part of the course?

- Writing performant code (best practice)
- The way to collaborate (the R ecosystem)
- Combines code, data and analysis
- Easy to distribute and re-use (public API)
- You learn how to re-use code from other packages

DESCRIPTION

DESCRIPTION

NAMESPACE

DESCRIPTION

NAMESPACE

R/
man/

DESCRIPTION

NAMESPACE

R/
man/

vignettes/
tests/
data/
src/
inst/

Creating a package

```
usethis::create_package("~/mypkg")
usethis::use_git()
usethis::use_github()
usethis::use_mit_license()
usethis::use_testthat()
usethis::use_package("ggplot2") # adds to Imports
usethis::use_r("my_function")   # creates R/my_function.R
```

usethis writes the boilerplate; you write the code.

```
Package: irisCourse
Type: Package
Title: Iris Course!
Version: 1.0
Author: Shashi Nagarajan
Maintainer: <shana92@liu.se>
Description: Visualises a scatter-plot of Sepal.Length vs.
  Petal.Width of Iris flowers in the iris dataset.
License: GPL-3 + file LICENSE
Encoding: UTF-8
Depends: R (>= 3.5.0)
Imports: ggplot2
Suggests: testthat (>= 3.0.0)
```

DESCRIPTION: Imports, Suggests, Depends (LABS!)

Your package will nearly always use functions from other packages.

Imports: must be present when your package is installed.

`library(yourpackage)` will *load* (not *attach*) them. Call them as `packagename::function()`.

Suggests: your package *can* use these, but does not require them – datasets, tests, vignettes. Check availability first with `requireNamespace()`.

Depends: these get *attached*. NOT recommended, heavy on the environment, CRAN limits the number. Use it to specify the R version.

H. Wickham, *R packages* (p. 34–36, 84)

What is used from other packages and what is provided to others.

- `importFrom(package, function)`: the package must be listed in DESCRIPTION. Do not `import(package)` (i.e. everything), import only what you need
- `export(function)`: what you make available to your users
- `exportPattern(regex)`: export everything matching a pattern
- `S3method(generic, class)`: export S3 methods
- `useDynLib`: import a function from C

H. Wickham, *R packages* (p. 84–90)

Do not call imported functions directly, call them through your own wrapper function.

1. Better control over what you pass – your wrapper's interface
2. If the foreign package or function gets deprecated you change it in only one place

Use `package::function()` for imported functions: it makes the code clearer and easier to update.

Why roxygen2?

R requires package documentation to be stored in `.Rd` files in the `man/` folder. These must conform to a number of specs, which is tedious to do by hand.

Enter **roxygen2**: document package elements in the `.R` files themselves and **roxygen2** generates `man/` for you.

Remember:

1. **roxygen2** is no free lunch, but it simplifies documentation – importantly it manages the NAMESPACE
2. Docs near the code help you keep them correct
3. Similar to JavaDoc and Doxygen
4. **roxygen2** needs **Encoding: UTF-8** in DESCRIPTION

Standard workflow:

1. Add comments to your `.R` files
2. Run `devtools::document()`
3. Preview the documentation and repeat as needed

Syntax:

1. All roxygen2 comments begin with the token `#'`
2. They are placed immediately before the function code
3. Most comments are written as tags, in the format `@tagname details`

Source: <https://r-pkgs.org/man.html>

```
#' Scatterplot of two features of the iris dataset  
##  
## Given a dataset like \code{iris}, \code{iris_plot}  
##      plots Sepal.Length vs. Petal.Width by Species  
##  
## @param data A \code{data.frame} with the continuous  
##      variables \code{Sepal.Length}, \code{Petal.Width}  
##      and the factor \code{Species}  
## @return A ggplot object  
## @export  
## @examples  
## iris_plot(iris)  
## @import ggplot2
```

The function itself follows on the next slide.

```
iris_plot <- function(data) {  
  ggplot2::ggplot(data) +  
    ggplot2::aes(x = Sepal.Length, y = Petal.Width,  
                 col = Species) +  
    ggplot2::geom_point(alpha = 0.5)  
}
```

Title, description and details are the first three blocks, separated by blank #' lines. Everything after that is tags.

`devtools::document()` writes both `man/iris_plot.Rd` and `NAMESPACE`:

```
# Generated by roxygen2: do not edit by hand
```

```
export(iris_plot)
import(ggplot2)
```

Never edit these two by hand once you use roxygen2 – your changes will be overwritten.

Tag	What it does
<code>@param</code>	Document one argument
<code>@return</code>	Document the return value
<code>@examples</code>	Runnable examples (are checked!)
<code>@export</code>	Add to NAMESPACE <code>export()</code>
<code>@importFrom</code>	Add to NAMESPACE <code>importFrom()</code>
<code>@field</code>	Document an RC/R6 field
<code>@references</code>	Literature, e.g. for the labs
<code>@noRd</code>	Document, but generate no <code>.Rd</code> file

Why unit testing?

- Fewer bugs
- Better code structure
- Faster restarts
- Robust code – correct a bug only once

A must in complicated projects!

The unit testing framework for R integrated with RStudio is `testthat`.

1. **White box** testing – you know the implementation and test its internal paths
2. **Black box** testing – you only know the interface, test what is promised
3. **Probabilistic** testing – the output is random, test its distributional properties

1. Set up the framework with `usethis::use_testthat()`
2. Write tests in `.R` files in `tests/testthat/`, the filename must start with `test` (e.g. `test-myfun.R`)
3. Run the tests with `Ctrl/Cmd + Shift + T` or `devtools::test()`

Test file structure:

1. Test files need `library`, `data` or similar statements if the tests need them
2. Test files have *test groups*, which contain *expectations* – the atomic unit of `testthat`
3. The `context()` statement is deprecated

Source: <https://r-pkgs.org/testing-basics.html>

```
data(iris)

test_that("Input must be a `data.frame`", {
  iris_matrix <- data.matrix(iris)
  expect_error(iris_plot(iris_matrix))
})
```

Expectations worth knowing

```
library(testthat)
test_that("euclidean() behaves", {
  euclidean <- function(a, b)
    if (b == 0) a else euclidean(b, a %% b)
  expect_equal(euclidean(100, 1000), 100)
  expect_true(is.numeric(euclidean(6, 4)))
  expect_error(euclidean("a", 1))
  expect_length(euclidean(6, 4), 1)
})
```

Test passed with 4 successes.

`expect_equal()` allows numerical tolerance, `expect_identical()` does not.

Turn bugs into test cases

```
my_median <- function(x) {  
  n <- length(x)  
  sort(x)[(n + 1) %/% 2]    # bug: even n  
}  
my_median(c(1, 2, 3, 4))    # should be 2.5  
  
## [1] 2
```

Turn bugs into test cases

```
my_median <- function(x) {  
  n <- length(x)  
  sort(x)[(n + 1) %/% 2]    # bug: even n  
}  
my_median(c(1, 2, 3, 4))    # should be 2.5
```

```
## [1] 2
```

Write the failing test *first*, then fix the code – now the bug can never come back unnoticed.

```
traceback()           # where did the last error happen?  
browser()             # drop into the debugger here  
debug(f); undebg(f)   # step through f every time it is called  
debugonce(f)  
options(error = recover) # pick a frame when an error occurs
```

Inside `browser()`: **n** next, **s** step in, **c** continue, **Q** quit, **where** show the call stack.

RStudio: click in the margin to set a breakpoint, then Ctrl/Cmd + Shift + L.

Checking the whole package

```
devtools::load_all()    # simulate library(yourpackage)
devtools::document()    # roxygen2 -> man/ + NAMESPACE
devtools::test()        # run testthat
devtools::check()       # R CMD check: the full battery
```

`devtools::check()` is what CRAN runs. Aim for **0 errors, 0 warnings, 0 notes**

ALWAYS CHECK INPUT!

Questions?

See you next time!